symmetry

MDPI

Article

Adaptive Exploration Proximal Policy Optimization for Efficient Robotic Continuous Control

Jiajian Li [1,2,*], Mingrui Li [2] and Hanshen Li [3]

[1] Goertek Inc., Nanjing 261031, China
[2] School of Information and Communication Engineering, Dalian University of Technology, Dalian 116024, China
[3] China Electric Power Planning & Engineering Institute, Beijing 100120, China; lhs@ncepu.edu.cn
* Correspondence: lijiajian_2023@163.com or 734225715@mail.dlut.edu.cn

Abstract

Proximal Policy Optimization (PPO) is widely adopted for robotic continuous control, yet it can suffer from insufficient exploration and unstable policy updates in high-dimensional action spaces. This paper proposes Adaptive Exploration Proximal Policy Optimization (AE-PPO), an enhanced PPO framework that integrates (i) adaptive clipping, which adjusts the clipping range according to the observed magnitude of policy updates to better balance stability and learning progress, (ii) adaptive entropy regularization, which schedules the entropy weight across training to maintain effective exploration while avoiding excessive randomness. AE-PPO is evaluated on standard MuJoCo continuous control benchmarks (e.g., Walker2d, HalfCheetah, and Humanoid) and compared with PPO and representative baselines such as Trust Region Policy Optimization (TRPO) and Soft Actor Critic (SAC). The results show that AE-PPO achieves faster convergence and an improved final performance with reduced training variance, demonstrating more stable and efficient learning in challenging high-dimensional tasks.

Keywords: reinforcement learning; PPO; adaptive exploration; robot control; continuous motion space

1. Introduction

Although Proximal Policy Optimization (PPO) has demonstrated strong stability in robotic control tasks, its fixed clipping threshold and static entropy regularization may limit its effectiveness in high-dimensional continuous control scenarios [1]. In complex environments such as humanoid locomotion, insufficient exploration and overly conservative policy updates can lead to slow convergence and an increased risk of suboptimal policies [2]. In addition, conventional training strategies may not fully utilize informative samples during learning, which can result in unstable policy updates and performance fluctuations during training [3]. These challenges motivate the development of adaptive mechanisms that can dynamically adjust key training parameters to improve both exploration efficiency and policy stability. Robotic continuous control tasks often involve high-dimensional state spaces and complex dynamic constraints. Traditional control methods struggle to adapt to such environments, making reinforcement learning particularly attractive for learning flexible control policies.

However, existing PPO-based approaches still face challenges in balancing exploration efficiency and training stability in complex robotic environments.

Academic Editor: Sergei Odintsov

Received: 5 February 2026
Revised: 18 March 2026
Accepted: 25 March 2026
Published: 24 April 2026

To address these issues, this paper proposes an improved reinforcement learning framework, AE-PPO (Adaptive Exploration Proximal Policy Optimization). The proposed method introduces two adaptive mechanisms. First, an adaptive clipping strategy dynamically adjusts the clipping range ε according to the observed magnitude of policy updates, enabling a better balance between convergence speed and policy stability [4]. Second, a stage-dependent entropy regularization mechanism is designed to adaptively regulate the entropy weight β(t), encouraging exploration during early training while gradually emphasizing policy refinement at later stages [5].

Despite the success of PPO in continuous control tasks, several limitations remain. First, the fixed clipping range may lead to either overly conservative updates or unstable policy changes in complex environments. Second, exploration efficiency is often limited by static entropy regularization. Third, the lack of adaptive parameter coordination may hinder stable learning across different training stages.

To evaluate the effectiveness of the proposed approach, experiments are conducted on multiple continuous control benchmarks in the MuJoCo simulation environment, including Walker2d, HalfCheetah, and Humanoid. AE-PPO is compared with several representative reinforcement learning algorithms, including PPO, TRPO, and SAC. Performance is evaluated using metrics such as convergence efficiency, reward stability, and task success rate. All experiments are conducted under a unified hardware configuration using NVIDIA RTX 3090 GPUs, and each task is executed with multiple independent random seeds to ensure statistical reliability.

The remainder of this paper is organized as follows. Section 2 reviews the theoretical background of PPO and related reinforcement learning approaches for continuous control. Section 3 presents the proposed AE-PPO framework and its adaptive mechanisms. Section 4 reports the experimental evaluation and ablation studies. Finally, Section 5 concludes the paper and discusses potential future research directions.

2. Technical Theory Research

2.1. PPO and Its Variants

2.1.1. Principle and Optimization Objectives of Classical PPO Algorithm

The Proximal Policy Optimization algorithm is a widely used policy optimization algorithm in reinforcement learning (Figure 1). It improves learning efficiency and stability while avoiding the common instability issues found in traditional policy gradient algorithms through the principle of "near-end optimization." PPO primarily achieves stable policy optimization by modifying the original policy gradient update rules [6].

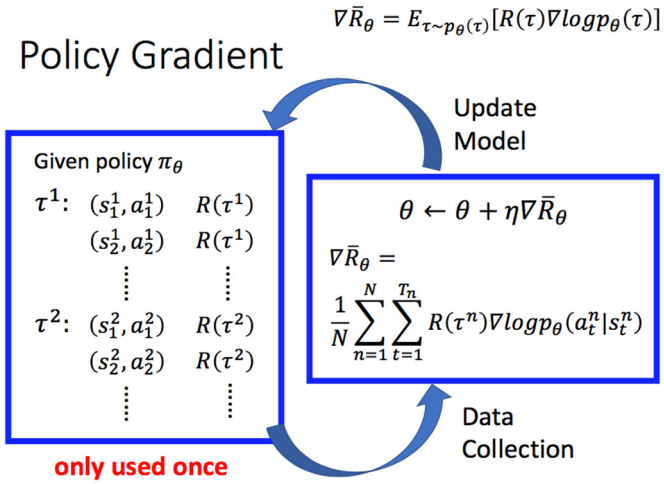


Figure 1. PPO algorithm.

In recent years, data-driven adaptive control methods, including event-triggered adaptive dynamic programming and fault-tolerant control for complex networked systems, have also attracted increasing attention in intelligent control research.

PPO is based on the idea of the cutting objective function, which aims to avoid large step changes during strategy updates and ensure the stability of the optimization process. Its core idea is to limit the step length of each update during strategy updates to prevent excessive updates from causing the strategy to shift too far [7].

Assuming the current π_θ strategy is, after one optimization $\pi_{\theta'}$, the strategy it becomes, PPO optimizes the strategy by using a weighted balancing term called the objective function. The objective function consists of the following two parts:

(1) Original objective function:

$$L^{CLIP}(\theta) = E_t[\min(\rho_t(\theta)A_t, clip(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)] \quad (1)$$

Among them $\rho_t(\theta)$, is the ratio of current strategy to old strategy, defined as

$$\rho_t(\theta) = \frac{\pi_\theta(\partial_t|S_t)}{\pi_{\theta old}(\partial_t|S_t)} \quad (2)$$

Here, the A_t estimated advantage ε function is a hyperparameter that controls the range of the shear.

The magnitude of policy update is quantified using the KL divergence between the updated policy and the previous policy.

(2) Cutting objective function: This objective function $\rho_t(\theta)$ avoids $1+\varepsilon 1-\varepsilon$ excessive updates by limiting the ratio within a range. The purpose of this is to prevent excessive changes in the strategy from affecting the stability of the update.

By minimizing this objective function, PPO ensures the stability of updates and can efficiently optimize policies. The optimization goal of PPO is to maximize the expected cumulative reward, while avoiding issues such as gradient explosion or unstable policy updates by limiting the scope of policy updates. Therefore, PPO primarily optimizes policies by balancing exploration and exploitation, achieving efficient learning and stable convergence [8]. The training process of PPO is shown in Figure 2.

In the PPO algorithm, the model first needs to learn through a trial-and-error process. Specifically, in the early stages, the model may take some less effective actions, resulting in lower rewards. However, this trial-and-error process is a crucial part of learning. The model gradually accumulates experience and learns how to make more favorable decisions to achieve higher rewards. PPO calculates rewards based on the model's performance in each round of trials. The size of the reward directly reflects the model's performance under the current decision: when the model performs well, the reward is high; conversely, if the decision is poor, the reward is low. In this way, PPO provides clear feedback to the model, helping it adjust its decision-making strategy.

According to the feedback from rewards, PPO gradually updates the model's strategy, which means adjusting decision-making rules to encourage the model to make more effective choices. For example, if a specific action yields high rewards, PPO increases the probability of selecting that action, guiding the model to repeat it more often. A core feature of PPO is the limitation on the magnitude of strategy updates. If each update was too large, it could lead to instability in the model or even make the learning process uncontrollable. To avoid this, PPO introduces a clipping mechanism that ensures each strategy update is not overly aggressive, maintaining stability and reliability during training. Through this mechanism, PPO can ensure efficient learning while avoiding excessive adjustments, ensuring steady optimization of the strategy.

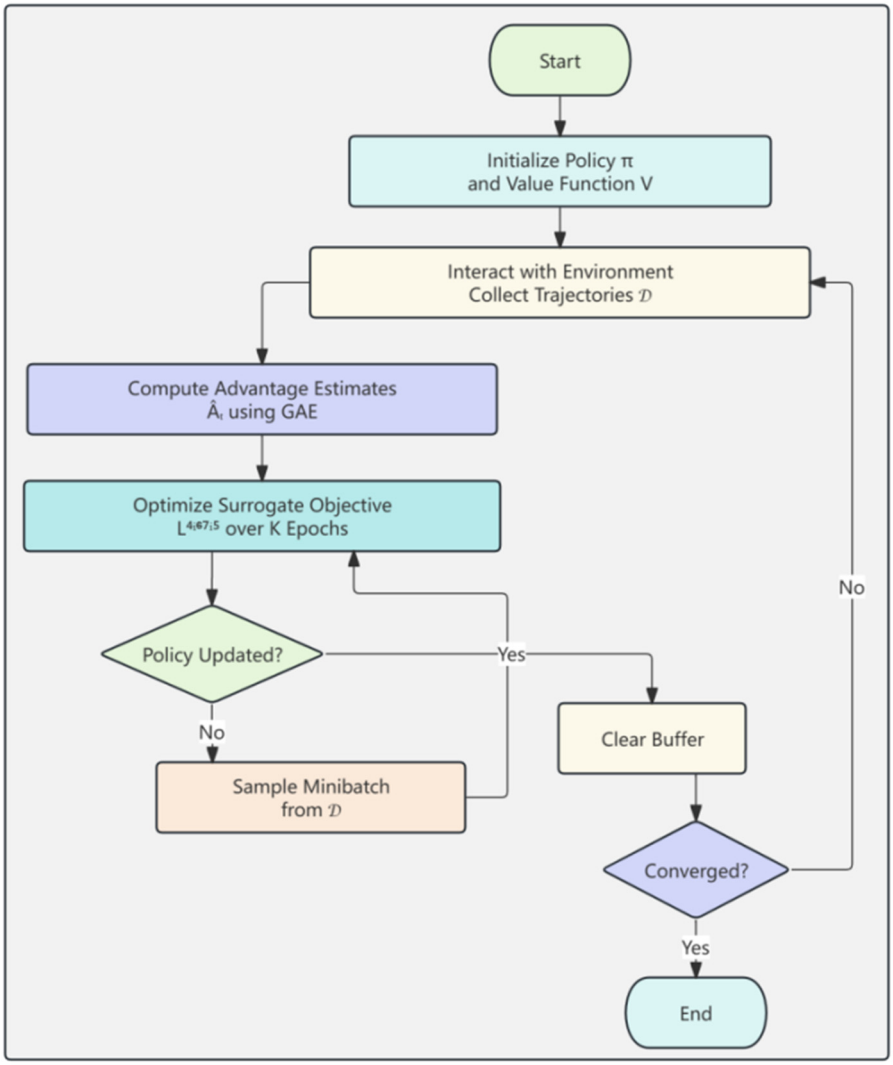


Figure 2. PPO training process.

2.1.2. Existing PPO Variants

In order to further improve the performance of the PPO algorithm, researchers have proposed a variety of PPO variants, which mainly optimize the performance of PPO in some specific tasks.

(1) Adaptive PPO: Adaptive PPO (A-PPO) adapts the clipping threshold ε to make policy updates more flexible. A-PPO dynamically adjusts ε based on the progress of current training ε. If the policy update is relatively stable during the current ε training phase, it increases to accelerate convergence; if instability occurs during training, it decreases to maintain stability. Specifically, the optimization objective of A-PPO is

$$L^{A-PPO}(\theta) = E_t[\min(\rho_t(\theta)A_t, clip(\rho_t(\theta), 1-\varepsilon_t, 1+\varepsilon_t)A_t)] \quad (3)$$

Among them ε_t, the parameters are dynamically adjusted based on the training errors of that period. A-PPO has strong adaptability during the training process, allowing for faster convergence and reducing the workload of manual parameter ε_t tuning. However, the adaptive adjustment mechanism may introduce additional computational overhead. In some extreme cases, adjustments can lead to instability.

(2) Trust Region PPO: Trust Region PPO (TRPO) is a PPO variant based on trust regions. TRPO controls the update step size of the policy by constraining the KL divergence

during each policy update, preventing excessive policy updates. The optimization objective of TRPO is

$$L^{TRPO}(\theta) = E_t[\min(\rho_t(\theta)A_t]subject\ to\ E_t[KL(\pi_{\theta old}||\pi_\theta)] \leq \delta \quad (4)$$

In Formula (4), δ is a trust region threshold serving as a hyperparameter, which limits the maximum value allowed for the KL divergence between the old and new policies. Among these $KL(\pi_{\theta old}||\pi_\theta)$, the differences between δ control KL strategies represent the maximum tolerated divergence. TRPO ensures stability during strategy updates through trust region constraints, reducing the risk of drastic updates. However, it has high computational complexity and requires solving complex constraint optimization problems. The training process can be slow, especially in large-scale problems.

(3) PPO with Curvature: PPO with Curvature is another variant of PPO that introduces curvature information to optimize the direction of policy updates. By considering the curvature of the objective function, PPO with Curvature improves upon the original gradient estimation method, making policy updates more efficient. The optimization objective is

$$L^{PPO-Curvature}(\theta) = E_t[\rho_t(\theta)A_t + \gamma_t {A_t}^2] \quad (5)$$

Among them γ_t, the curvature parameter, controls the curvature information of the advantage function. After introducing curvature information, it can better capture the sensitivity of strategy updates, improving learning efficiency. It can achieve good results in complex tasks. However, it requires additional calculation of the second-order information of the advantage function, increasing computational complexity.

2.2. Application of Reinforcement Learning in Continuous Motion Space Robot Control

MuJoCo is a widely used high-performance physics engine designed for robotic control, kinematics, and dynamic simulation. It provides accurate rigid-body dynamics modeling and efficiently handles complex physical interactions such as contact and friction, making it a standard simulation platform for robotics and reinforcement learning research [9]. Through precise physical modeling and efficient numerical computation, MuJoCo enables researchers to develop and evaluate control algorithms in complex simulated environments.

The platform provides a series of standardized benchmark tasks that are widely used to evaluate reinforcement learning algorithms in continuous control problems. These tasks include locomotion, balancing, and object manipulation scenarios, which require agents to learn effective control strategies in high-dimensional state–action spaces. Such benchmark environments allow for the systematic comparison of different reinforcement learning methods and provide a reliable framework for analyzing algorithm performance and robustness under complex dynamic conditions [10].

In addition, MuJoCo is optimized for fast physical simulation, enabling efficient large-scale training and experimentation in reinforcement learning studies [11]. By leveraging these advantages, researchers can design controlled experimental environments to investigate the behavior and performance of learning-based control algorithms in robotic systems. Figure 3 illustrates the general simulation framework of the MuJoCo-based robotic control environment used in this study.

In complex robotic environments, agents must make sequential decisions under uncertainty and dynamic environmental changes. Traditional robot control approaches typically rely on precise system modeling and predefined control rules, which may limit their adaptability in highly dynamic scenarios. In contrast, reinforcement learning provides

a data-driven framework that enables agents to learn optimal control policies through interaction with the environment.

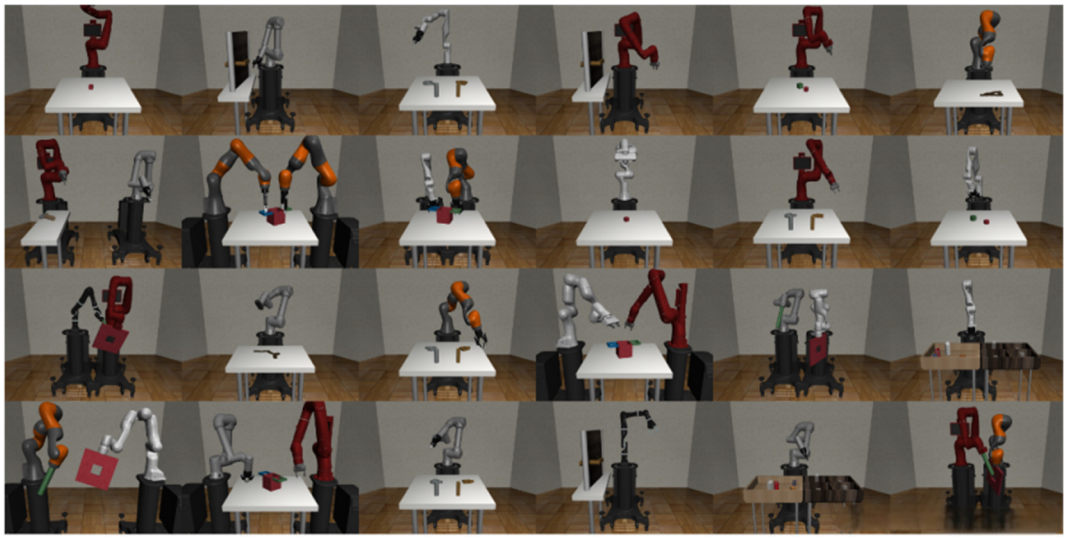


Figure 3. Schematic diagram of robot simulation platform.

Through trial-and-error learning guided by reward signals, reinforcement learning algorithms can gradually improve policy performance and discover effective control strategies without requiring an explicit model of the environment. This capability makes reinforcement learning particularly suitable for complex decision-making tasks such as dynamic obstacle avoidance, robotic manipulation, and autonomous locomotion. By continuously updating policies based on environmental feedback, reinforcement learning agents can achieve improved adaptability, robustness, and task efficiency in complex robotic control scenarios.

2.3. Related Reinforcement Learning Optimization Strategies

Reinforcement learning (RL) often faces challenges such as exploration and exploitation trade-off, low sample efficiency and slow convergence speed when training agents. In order to solve these problems, many optimization strategies have emerged. The following are some commonly used optimization strategies: entropy regularization, experience replay, etc.

(1) Entropy regularization improves exploration ability

Entropy regularization is a strategy used to enhance exploration capabilities. It encourages agents to take more diverse actions by adding an entropy term to the loss function, rather than getting stuck in local optima. The core idea of entropy regularization is to increase the uncertainty of strategies, prompting agents to explore more strategies during training, instead of relying solely on already-discovered effective ones, as shown in Figure 4.

For the strategy π, entropy regularization is usually expressed as

$$L_{entropy} = -\beta E_{\pi}[\log \pi(\partial_t | S_t)] \tag{6}$$

Among them β, the regularization coefficient, controls $\pi(\partial_t|S_t)$, the S_t rate of entropy ∂_t regularization and the probability of taking actions in a state. By maximizing this entropy term, the strategy will tend to become more evenly distributed, thus increasing exploration and encouraging the agent to explore more action spaces, avoiding premature convergence.

This prevents the model from relying solely on a few strategies, thereby reducing the risk of overfitting.

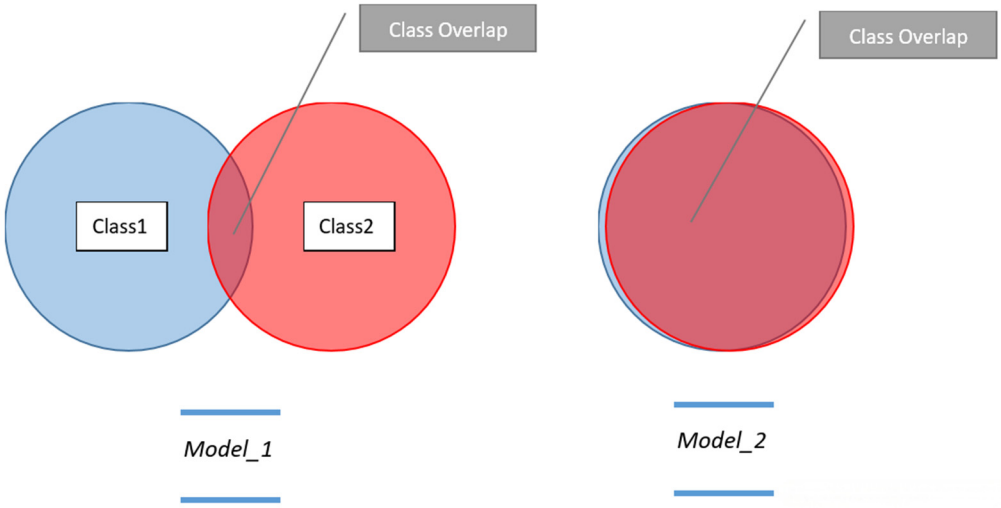


Figure 4. Entropy regularization and entropy minimization.

(2) Experience playback improves data utilization

Experience replay is a method to improve the efficiency of sample utilization. By storing the interaction experiences of agents in a buffer, agents can randomly draw experiences for training, rather than relying solely on current experiences. This helps break down correlations between data, reduce high variance, and enhance training stability, as shown in Figure 5.

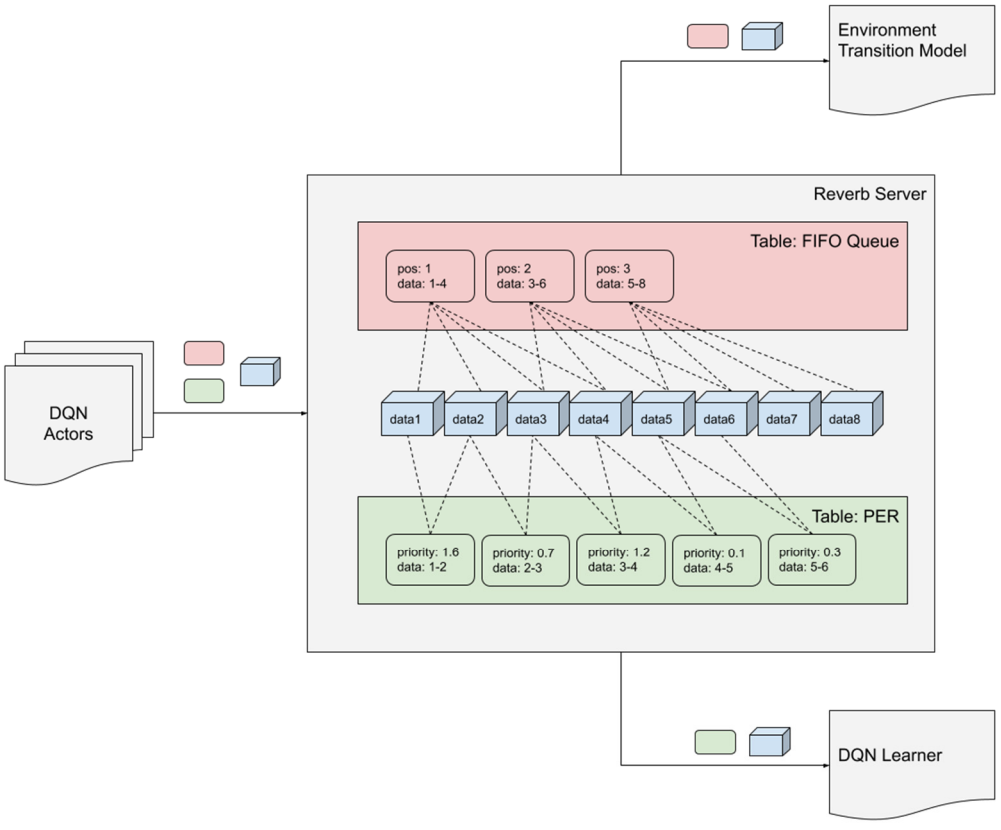


Figure 5. Experience playback framework.

Assume that the experience D pool of the agent stores $(S_t, \partial_t, r_t, S_{t+1})$ past experience samples. In D for each training step, the agent $\{(S_i, \partial_i, r_i, S_{i+1})\}$ randomly selects a small batch of experience from it and trains it. The objective function is usually

$$L_\theta = E_{(S,\partial,r,S')\sim D}[(r + \gamma Q(S', a') - Q(S, a))^2] \quad (7)$$

Among them γ is the discount $Q(S, a)$ factor and the action value function. By reusing experience, the utilization of data is significantly improved. In Formula (7), γ represents the standard discount factor, which is used to weigh the current value of future rewards. The correlation of data in the training process is reduced and the oscillation of strategy is avoided.

3. Improved PPO Algorithm

3.1. Improvement Strategies

This paper proposes four collaborative improvement strategies to address the limitations of the traditional PPO algorithm in continuous robot control. Firstly, an adaptive clipping mechanism that dynamically adjusts the clipping range is introduced to solve the training oscillation problem caused by a fixed clipping threshold. Secondly, an entropy weight adjustment method based on the training stage is designed to balance the exploration and exploitation conflict. Furthermore, a priority experience replay strategy is introduced to enhance the utilization rate of high-value samples. Finally, the learning rate is automatically adjusted based on the magnitude of policy updates to enhance training stability. These four improvements achieve joint optimization of efficiency and stability through parameter linkage.

The four adaptive mechanisms operate in a coordinated manner. The adaptive clipping mechanism controls the step size of policy updates, ensuring stable policy optimization. The entropy adjustment mechanism regulates exploration intensity during different training stages. The prioritized replay mechanism improves the utilization of high-value samples, accelerating learning efficiency. Meanwhile, the adaptive learning rate dynamically adjusts the optimization step size according to gradient stability. Together, these mechanisms form a coordinated parameter linkage that balances exploration efficiency and training stability throughout the learning process.

3.1.1. Adaptive Clipping (Adaptive Clipping)

In the standard PPO algorithm, the clipping operation plays a critical role in constraining the magnitude of policy updates and stabilizing the optimization process. By restricting the probability ratio within a predefined interval, the clipping mechanism prevents excessively large policy updates that may destabilize training. In practice, PPO typically adopts a fixed clipping threshold (e.g., ε = 0.2) that is applied uniformly across all training iterations [12].

However, a fixed clipping range may not be optimal throughout the entire training process, particularly in high-dimensional and complex control tasks. If the clipping threshold is set too conservatively, policy updates may become overly restricted, leading to slow learning and a reduced exploration capability. Conversely, an excessively large clipping range may allow overly aggressive updates, potentially causing unstable policy learning. Therefore, introducing an adaptive clipping mechanism that dynamically adjusts the clipping range according to the magnitude of policy updates can provide a more flexible balance between stability and learning efficiency.

To address the issue of insufficient flexibility of the fixed clipping range ε in complex tasks, AE-PPO introduces an adaptive clipping mechanism. The core of this mechanism is to

dynamically adjust the ε value based on the consistency of policy updates [4]. Specifically, we calculate the KL divergence between the current policy and the old policy at the end of each training batch as a measure of the "update magnitude". To smooth out fluctuations, this value is processed using exponential moving averages. Subsequently, ε is dynamically adjusted based on the ratio of the smoothed update magnitude to its historical maximum value: when the update magnitude is small (indicating a smooth change in policy), ε is appropriately increased to accelerate convergence; when the update magnitude approaches historical peaks (indicating drastic changes in policy), ε is reduced to maintain training stability. The initial ε is set to 0.2, and its adjustment range is controlled by a proportional coefficient to ensure smooth changes. This method enables the algorithm to automatically adapt to the needs of different training stages, eliminating the need for the manual adjustment of fixed parameters.

Assume θ is the current θ' strategy parameter and $r(\theta)$ is the updated strategy parameter, which represents the probability ratio of the current strategy. The traditional clipping operation is

$$L_{clip}(\theta) = E_t[\min(\rho_t(\theta)A_t, clip(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)] \tag{8}$$

Among ε, in order to fix the clipping range, in adaptive ε clipping, the $\varepsilon_{adaptive}(t)$ clipping range becomes dynamic, which is denoted as and updated by the following rules:

$$\varepsilon_{adaptive}(t) = \varepsilon_{inital} \cdot (1 + \alpha \cdot \frac{UpdateMagnitude(t)}{MaxUpdateMagnitude}) \tag{9}$$

Among them, ε_0 is the initial clipping range, α is the adjustment parameter, δ_t is the current step strategy update amplitude, δ_{max} is the maximum update amplitude in the whole training process.

3.1.2. Entropy Regularization Exploration (Entropy-Based Exploration)

In reinforcement learning, the diversity of strategies and exploration capabilities are crucial for avoiding local optima. Entropy regularization enhances exploration by introducing an entropy term to encourage strategic randomness [13]. Specifically, entropy regularization adds a term related to the entropy of strategy distribution to the objective function, forcing the agent to maintain high strategic uncertainty during learning. This strategy effectively prevents the agent from converging too early to suboptimal strategies, thereby enhancing the comprehensive exploration of the environment, especially in complex or uncertain settings. Traditional entropy regularization methods typically use a fixed entropy weight to control the guiding effect of entropy, but this fixed weight may produce mismatched results at different training stages [6].

To better adapt to the needs of different training stages, this paper proposes a dynamic entropy weight calculation method. The adaptive adjustment of entropy-regularized weight β aims to balance exploration and exploitation. AE-PPO adopts a two-stage adjustment strategy. Firstly, a linear decay plan based on training progress is set, gradually reducing β from its initial value of 0.05 to 0.005, ensuring that the focus is on policy optimization in the later stages of training. Secondly, real-time feedback of policy entropy is introduced for fine-tuning: in each iteration, the average entropy value of the current policy is calculated. If the entropy value is below a preset target threshold (indicating that exploration may be insufficient), β is temporarily increased; if the entropy value is too high (potentially leading to excessive randomness), β is decreased. This feedback mechanism draws on the idea of target entropy, but integrates it into the dynamic modulation of weights, allowing the exploration intensity to respond to the actual maturity of the policy, rather than simply changing over time.

The entropy weight $\beta(t)$ here is a variable that is dynamically adjusted with the training stage. It has the same symbol as the entropy regularization coefficient β appearing as a fixed hyperparameter in Formula (6) in Section 2.3, but its meaning is different. In this algorithm, the value of $\beta(t)$ is adaptively adjusted based on the training progress and policy entropy.

$$L_{total}(\theta) = E_t[L_{clip}(\theta) + \lambda(t)H(\pi(\cdot|S_t))] \quad (10)$$

Among $L_{clip}(\theta)$, the traditional cutting $H(\pi(\cdot|S_t))$ objective function is $\lambda(t)$ which is used for the entropy of the strategy distribution, and the entropy weight is dynamically adjusted. This method of dynamically adjusting the entropy weight can effectively enhance the exploratory nature of the agent in the early stages of training, while avoiding over-reliance on randomness in the later stages. As a result, the training process can ensure efficient learning while maintaining sufficient exploration.

3.1.3. Enhanced Experience Playback

In reinforcement learning, experience replay is widely used to improve sample efficiency and stabilize the training process by reducing the correlation among collected samples. By storing agent–environment interactions in a replay buffer and sampling mini-batches for training, the algorithm can reuse historical experiences and enhance data utilization. However, conventional replay mechanisms typically treat all experiences equally, which may cause informative samples to be underutilized during learning. To address this limitation, prioritized experience replay (PER) has been proposed to improve learning efficiency by assigning higher sampling probabilities to more informative transitions [14].

The PER mechanism evaluates the importance of each transition based on its temporal difference (TD) error. Transitions with larger TD errors are considered more informative and are therefore sampled more frequently during training. This strategy allows the learning process to focus on critical state–action pairs that contribute more significantly to policy improvement, thereby accelerating convergence and improving learning efficiency [15].

To integrate prioritized replay within the AE-PPO framework while maintaining compatibility with PPO's near-on-policy training paradigm, several modifications are introduced. First, the replay buffer stores only data from the most recent training iterations, ensuring that sampled experiences remain relevant to the current policy. Second, sampling priorities are computed using the absolute TD error of each transition. Finally, importance-sampling weights are applied to correct the bias introduced by non-uniform sampling, allowing for the unbiased estimation of the policy gradient during optimization.

Through this design, AE-PPO emphasizes informative experiences while preserving training stability. The prioritized sampling strategy improves the utilization of high-value samples and enhances overall learning efficiency in complex robotic control tasks.

3.1.4. Adaptive Learning Rate Adjustment

In reinforcement learning, the learning rate is a key hyperparameter that significantly influences both convergence speed and training stability. An excessively large learning rate may cause unstable updates, while a small learning rate may slow down convergence and reduce training efficiency [16]. Figure 6 illustrates the dynamic adjustment of the learning rate. Therefore, dynamically adjusting the learning rate during training has been widely studied as an effective strategy for improving optimization performance.

To address this issue, this paper introduces an adaptive learning rate adjustment mechanism that modifies the learning rate according to the magnitude of policy updates during training [11]. Instead of using a fixed learning rate throughout the entire optimization process, the proposed strategy adjusts the step size in response to the observed training dynamics, allowing the algorithm to better balance convergence efficiency and stability.

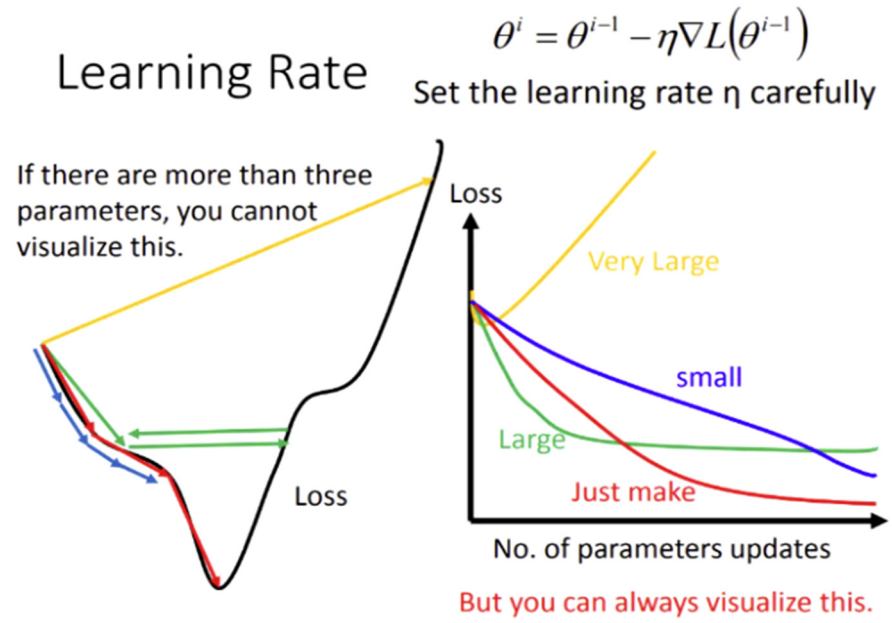


Figure 6. Adaptive learning rate adjustment.

The core idea of the adaptive learning rate mechanism is to regulate the update step size based on feedback from the training process. During the early stages of training, when the policy is still far from convergence, a relatively larger learning rate is adopted to accelerate policy improvement. As training progresses and policy updates become more stable, the learning rate is gradually reduced to avoid excessive updates and ensure stable convergence. This adaptive strategy enables the training process to maintain efficient learning while preventing instability caused by overly aggressive updates [17].

Furthermore, the adaptive learning rate mechanism dynamically responds to variations in policy update magnitude during optimization. When large fluctuations in policy updates are detected, the learning rate is automatically reduced to stabilize gradient updates. Conversely, when the policy update becomes relatively stable, the learning rate is moderately increased to maintain sufficient learning progress. Through this feedback-driven adjustment, the algorithm can achieve a more stable and efficient training process in complex control tasks [18].

3.2. Algorithm Process

During reinforcement learning training, the quality and diversity of sampled data significantly influence the effectiveness of policy optimization. Efficient data sampling mechanisms help agents sufficiently explore the state–action space while improving the utilization efficiency of collected experiences. To improve training efficiency and data quality, this study designs an optimized data sampling and processing workflow within the AE-PPO framework [19].

In the early stages of training, exploration is encouraged through stochastic policy sampling, allowing the agent to interact with the environment and collect diverse state–action transitions. Such stochastic sampling helps prevent premature convergence to suboptimal solutions and promotes broader exploration of the environment [20].

To further improve training efficiency, prioritized sampling is incorporated into the experience replay mechanism. In this strategy, transitions with larger temporal difference (TD) errors are assigned higher sampling probabilities, as these samples typically contain more informative learning signals. By focusing more frequently on high-error transitions, the algorithm can accelerate policy improvement and enhance learning efficiency [21].

An experience replay buffer is used to store recent interaction experiences generated during training. Mini-batches are sampled from the replay buffer to update the policy network, which reduces the correlation among consecutive samples and improves the stability of the learning process. Through the combination of stochastic exploration, prioritized sampling, and experience replay, the proposed data sampling workflow improves data utilization efficiency while maintaining stable policy optimization.

During the training process, we adjust the updated gradient based on sample weights. If a sample has a higher weight (i.e., it has a greater impact on the current strategy), its contribution to the gradient will also increase, for example, in one update, sample (0.5, 0.3) has a weight of 0.2 and sample (0.2, 0.7) has a weight of 0.5. When updating the strategy, samples with higher weights will have a greater influence on the strategy update. The final data sampling and processing results are shown in Table 1.

Table 1. Final data sampling and processing effect.

State (s)	Movement (a)	Reward I	Error (δ)	Weight (w)	Frequency of Sampling
(0.5, 0.3)	take up	10	0.2	0.2	0.25
(0.2, 0.7)	lay down	5	−0.5	0.5	0.4
(0.3, 0.6)	take up	8	0.1	0.1	0.15
(0.8, 0.2)	lay down	6	0.3	0.3	0.2

In reinforcement learning, the strategy update is a crucial step in the learning process, which directly affects the performance and learning efficiency of agents. In order to ensure an effective strategy update, especially in complex tasks, it is necessary to consider the optimization of the strategy update to improve the convergence speed and stability.

The process of strategy updating primarily comprises the following steps:

(1) Sampling and data accumulation: The agent interacts with the environment (S_t, ∂_t) according to the current strategy, accumulates state–action pairs and corresponding reward information (r_t). These data are usually stored in the experience replay pool for subsequent training.

In the strategy update, the sampling experience is used to calculate the advantage function and the value function.

(2) Calculate the objective function: The objective function is used to measure the gap between the current strategy and the target strategy. In PPO, the objective function is typically the clipped policy loss, which aims to avoid excessive updates by controlling the step size of policy updates, thus maintaining the stability of the training process. The objective function for PPO can be expressed as

$$L^{CLIP}(\theta) = E_t[\min(\rho_t(\theta)A_t, clip(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)] \quad (11)$$

(3) Calculate the gradient and update: Use the gradient descent method (θ) to calculate the gradient of the objective function with respect to the policy parameters. Optimize the policy parameters using an optimization algorithm to minimize the objective function, thereby improving the policy. In PPO algorithms, gradient updates are typically performed using mini-batch stochastic gradient descent to reduce the impact of each update on stability.

(4) Constraint: In order to avoid large steps in each update, PPO uses the clipping mechanism. This mechanism limits the range of policy updates to ensure that the policy does not deviate too far from the current policy, thus controlling the stability of policy updates.

(5) Updated strategy evaluation: After each strategy update, the performance of the new strategy in the current environment needs to be evaluated. The benefits of the strategy can be calculated through multiple interactions with the environment, and the advantages and disadvantages of the strategy can be evaluated.

3.3. Pseudo Code

In this section, we will show the pseudo code of the PPO algorithm based on adaptive clipping, entropy regularization, experience replay, and learning rate adjustment. This framework integrates the above improvement strategies to make policy updates more stable, have faster convergence, and improve learning efficiency, as shown in Table 2.

Table 2. Core framework of algorithm implementation.

Step	Core Code
1	experiences = collect_trajectories(env, policy, n_steps)
2	td_error = compute_td_error(value_net, s, r, s', gamma)
3	batch = replay_buffer.sample_prioritized(td_error, batch_size)
4	ratio = exp(new_log_prob-old_log_prob); clipped_ratio = clip(ratio,1 − ε_adaptive,1 + ε_adaptive)
5	entropy_term = −β_dynamic * entropy(policy_distribution)
6	loss = clipped_policy_loss + entropy_term; optimizer.step()
7	lr = adjust_lr(lr, iteration, update_magnitude); optimizer.param_groups[0]['lr'] = lr

Note: ε_adaptive denotes the dynamically adjusted clipping range; β_dynamic is the adaptive entropy weight; policy_distribution refers to the current policy's action distribution. The code snippets reflect the core logic of AE-PPO's key modules as described in the manuscript.

The adaptive clipping mechanism calculates the ratio between the current strategy and the old strategy, i.e., ratio = exp(new_log_prob-old_log_prob) and clips this ratio using clip (ratio, 1 − epsilon, 1 + epsilon). This ensures that the magnitude of strategy updates is not too large, thus avoiding instability caused by excessive updates [22]. This strategy helps control the pace of strategy updates, ensuring each update remains within a reasonable range. Additionally, to enhance the agent's exploration capabilities and encourage diversity in strategies, an entropy regularization term entropy_term = −entropy_weight * entropy($\pi(\theta)$) is introduced. By increasing entropy, it prevents strategies from converging too quickly, allowing the agent to explore more of the state space. To improve training efficiency and stability, experience replay and batch update methods are also adopted. Within each training cycle, the experience replay pool is used to break correlations between data, which helps improve data utilization efficiency and enhances training stability. Finally, through adaptive learning rate adjustment technology, adjust_learning_rate(optimizer, iteration) is used to dynamically adjust the learning rate during the training process, so as to avoid instability caused by a too large or too small learning rate, and ensure that the model can adapt to the appropriate pace in different training stages, optimizing the effect of the training process.

4. Experiments and Results

4.1. Experimental Setup and Evaluation Protocol

To ensure experimental reproducibility, reliability, and fair comparison with existing studies, this section describes the experimental environment, training protocol, and evaluation criteria in detail.

Hardware and Software Environment.

All experiments were conducted on a workstation equipped with an NVIDIA RTX 3090 GPU. The proposed algorithm was implemented using the PyTorch 1.12.1 deep learning framework. The simulation environment was built on the MuJoCo 2.2.2 physics engine, combined with the Gymnasium 0.28.1 reinforcement learning benchmark library. Standard continuous control tasks were adopted, including Walker2d-v4, HalfCheetah-v4, Humanoid-v4, Ant-v4, Hopper-v4, and HumanoidStandup-v4. All tasks use the latest stable version (v4) to ensure consistency in the evaluation environment.

Training and Evaluation Protocol.

To evaluate the robustness of the proposed algorithm, each task was trained using five independent random seeds, controlling factors such as network initialization and environment stochasticity. During training, the policy was evaluated every 10,000 environment interaction steps. For evaluation, a deterministic policy was used (i.e., selecting the mean action output by the policy network) and executed for 10 complete episodes. The reported performance corresponds to the average cumulative reward across these episodes. All learning curves represent the mean performance over the five seeds, while the shaded regions indicate ± 1 standard deviation to illustrate the variability across different runs.

Performance Metrics and Statistical Testing.

In addition to learning curves, several quantitative metrics are used for final performance comparison:

(1) Final Average Reward, defined as the average reward across the last ten evaluation points;

(2) Convergence Speed, measured as the number of environment interaction steps required to reach 95% of the final average reward for the first time;

(3) Training Stability, quantified as the standard deviation of rewards during the final 100,000 training steps.

To assess statistical significance, the final average rewards obtained from five random seeds are collected for each algorithm. A paired t-test with significance level $\alpha = 0.05$ is conducted to determine whether performance differences between AE-PPO and baseline algorithms are statistically significant. All statements regarding performance improvement or statistical significance in this paper are based on these hypothesis testing results.

4.2. Introduction to Robot Task Environment

4.2.1. MuJoCo, Introduction of Simulation Platform

MuJoCo is an efficient physics simulation engine developed by Emo Todorov, widely used in robotics, control systems, and reinforcement learning. Its core strength lies in its powerful physics simulation capabilities, particularly excelling in the simulation of complex object interactions and collisions. MuJoCo uses advanced constraint optimization algorithms to quickly compute dynamic AE-PPO responses for multi-body systems, including contact, friction, and joint constraints, providing precise simulation support for various robotics and control system tasks. Additionally, MuJoCo supports accurate dynamics modeling, capable of simulating collisions between rigid bodies, joint movements, and complex mechanical interactions. Users can finely set physical properties such as mass, shape, and inertia of objects according to their needs, ensuring highly realistic simulation results.

The high customizability of MuJoCo is another significant advantage. It offers a wide range of configuration options, allowing users to tailor the simulation environment according to specific task requirements. For example, it supports various dynamic and friction models and allows parameters such as time step size to be adjusted, meeting the needs of different types of tasks. MuJoCo not only supports multi-platform operation, including Windows, Linux, and macOS operating systems, but also provides a Python 3.8 interface,

enabling seamless integration with reinforcement learning frameworks like gym in OpenAI Gym version 0.26.2 and Ray RLlib version 2.4.0. Due to its powerful simulation capabilities, MuJoCo is widely used in robotics, control, robotic arm manipulation, humanoid robots, and aircraft control, making it a key tool for studying issues such as robot control, motion planning, and multi-robot collaboration [23]. MuJoCo is also extensively applied in robotics, control, reinforcement learning, robotic arm manipulation, humanoid robots, and aircraft control. Its strong simulation capabilities make it an essential tool for researching issues related to robot control, motion planning, and multi-robot collaboration, as shown in Figure 7.

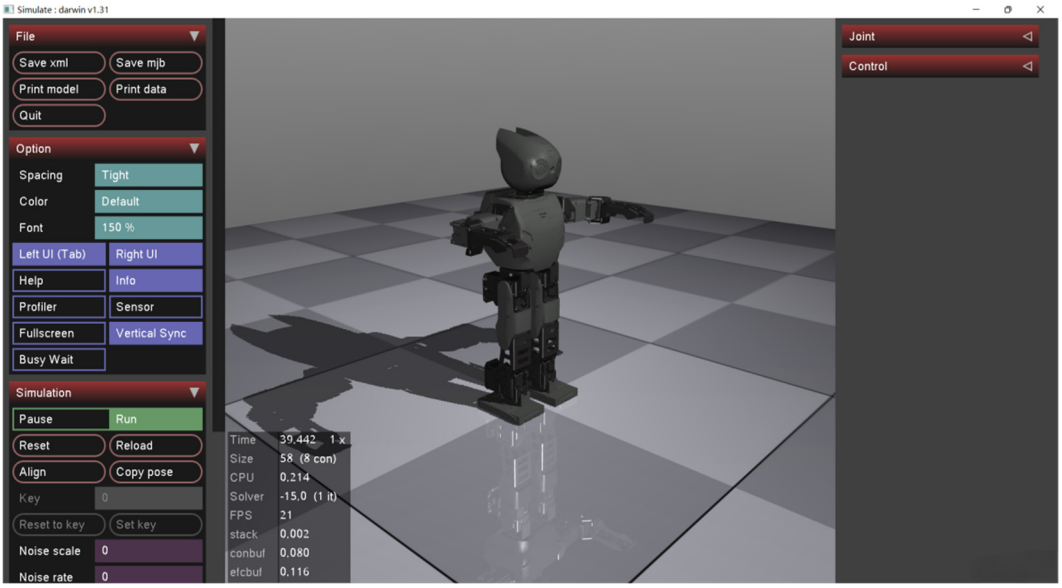


Figure 7. Key components of MuJoCo simulation platform.

4.2.2. Task Environment Design

In reinforcement learning, the configuration of the task environment directly affects the difficulty of the training process and the performance of the algorithm. To verify the effectiveness of the improved PPO algorithm in robot control tasks, we selected a classic task environment from the MuJoCo simulation platform. The specific task environment configuration includes the definition of the task objective, robot model, state space, and action space, as well as the design of the reward function. The configurations for different task environments are shown in Table 3.

Table 3. Configuration of different mission environments.

Task Environment	Task Object	State Space Dimensions	Action Space Dimension	Challenging
Reacher	Move the end effector of the robotic arm to the target position	9	4	Medium difficulty, the movement of the robot arm needs precise control
Pusher	Control the robotic arm to push the object to the target area	9	4	It is hard and you have to push the object multiple times and consider the reaction of the object
Ant	Make the quadruped robot walk and maintain balance	111	8	It is difficult and requires efficient motion control and balance strategies
Humanoid	Control humanoid robots to stand, walk or run	376	17	It is very difficult, it involves complex coordination and motion control

4.3. Experimental Results and Analysis

4.3.1. Convergence Curve Comparison

Table 4 shows the experimental results of three typical tasks. We collect the performance of different algorithms in the training process and compare them through multiple indicators.

Table 4. Comparative analysis of convergence curves of AE-PPO, PPO and TRPO.

Task	Algorithm	Final Average Reward	Number of Steps Required for Convergence	Training Stability	Task Success Rate
Walker2d	AE-PPO (Ours)	3150 ± 105	0.85 M ± 0.04 M	0.95 ± 0.03	98% ± 1.5%
PPO (Baseline)	2680 ± 135	1.20 M ± 0.07 M	0.82 ± 0.07	92% ± 3.0%	
TRPO	2450 ± 150	1.35 M ± 0.08 M	0.78 ± 0.08	88% ± 4.0%	
HalfCheetah	AE-PPO (Ours)	5850 ± 220	0.75 M ± 0.03 M	0.93 ± 0.04	100%
PPO (Baseline)	4950 ± 280	1.05 M ± 0.06 M	0.80 ± 0.09	100%	
SAC	6100 ± 195	0.90 M ± 0.05 M	0.96 ± 0.02	100%	
Humanoid	AE-PPO (Ours)	2750 ± 120	0.65 M ± 0.05 M	0.92 ± 0.05	92% ± 2.0%
PPO (Baseline)	2320 ± 165	1.00 M ± 0.08 M	0.75 ± 0.10	85% ± 3.5%	
TRPO	2200 ± 180	1.25 M ± 0.10 M	0.88 ± 0.06	83% ± 4.0%	

As shown in Table 4, AE-PPO demonstrates consistent performance improvements over the baseline PPO across all tasks. In the challenging Humanoid task, AE-PPO achieves a final average reward of 2750 ± 120, which represents an improvement of approximately 18.5% compared with PPO (2320 ± 165). In addition, AE-PPO shows better convergence efficiency, requiring fewer environment interaction steps to reach a stable performance. For example, in the Walker2d task, AE-PPO converges within 0.85 M steps, while PPO requires approximately 1.20 M steps.

In terms of training stability, AE-PPO also exhibits smaller fluctuations compared with PPO, indicating more stable policy updates during training. These results suggest that the adaptive mechanisms introduced in AE-PPO help improve optimization stability and learning efficiency.

For the HalfCheetah task, AE-PPO achieves faster convergence than PPO and requires fewer training steps than the off-policy baseline SAC, while maintaining a competitive final performance. Similarly, in the Humanoid task, AE-PPO consistently outperforms both PPO and TRPO in terms of convergence speed and final reward.

Overall, these results indicate that the proposed AE-PPO algorithm improves training efficiency and policy stability while maintaining a strong task performance across multiple continuous control environments.

According to the experimental results, as shown in Figure 8, it particularly excels in convergence speed, task success rate, training stability, energy consumption, and average steps.

4.3.2. Ablation Experiments

To investigate the individual contributions of each improvement component in the proposed AE-PPO framework, a series of ablation experiments were conducted on the Humanoid-v4 task. The four modules evaluated include adaptive clipping, entropy-adaptive exploration, prioritized experience replay with importance sampling correction, and adaptive learning rate adjustment.

In the ablation study, each module was individually added to the baseline PPO algorithm to evaluate its independent contribution. The performance was measured

using the average episode reward and convergence steps. The experimental results are summarized in Table 5.

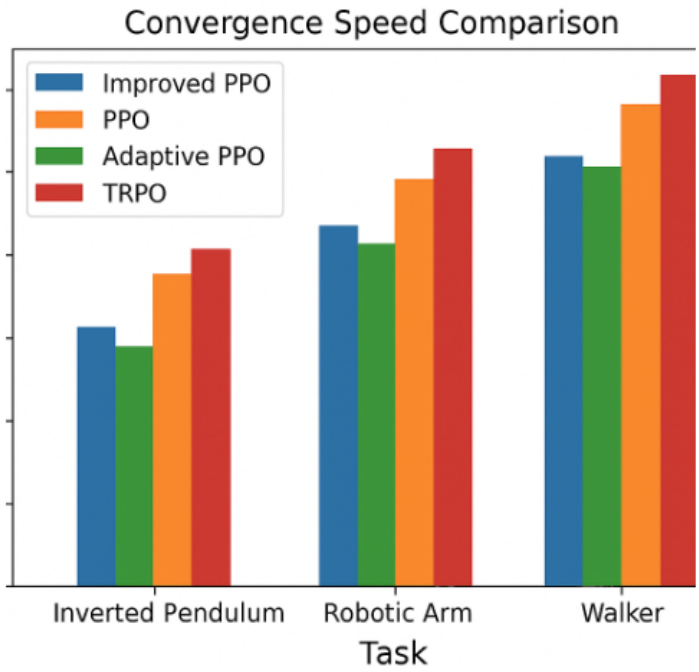


Figure 8. Comparison of different models.

Table 5. The independent effects of each improvement module are measured by the average reward.

Method	Average Award (Episode Reward)	Rapidity of Convergence (Steps)
Baseline-PPO	1850	1.0 M
+ Adaptive Clipping	2100	0.85 M
+ Entropy-based Exploration	2250	0.78 M
+ Improved Experience Replay	2350	0.75 M
+ Adaptive Learning Rate	2450	0.72 M

Table 5 shows that each improvement module provides a noticeable performance gain compared with the baseline PPO. In particular, the adaptive learning rate and prioritized experience replay modules lead to more significant improvements in both reward and convergence speed, indicating that these mechanisms effectively enhance optimization efficiency.

To further evaluate the joint contribution of all modules, we compare the full AE-PPO model (i.e., all four modules enabled) with several ablated variants in which one module is removed. The results are presented in Table 6. As shown in the table, the complete AE-PPO model achieves the best performance with an average reward of 2750 and convergence within 0.65 M steps. Removing any individual component results in performance degradation, suggesting that the proposed modules provide complementary benefits during policy optimization.

Figure 9 illustrates the corresponding performance trends for different ablation configurations.

The corresponding AE-PPO line graph is shown in Figure 9.

To further analyze the impact of each improvement module on training stability, we evaluate the reward variance during training, as shown in Table 7. Lower reward variance indicates more stable policy learning. The results demonstrate that the integration

of the proposed modules significantly reduces reward fluctuations compared with the baseline PPO.

Table 6. The effect of combining different strategies.

Method	Average Award (Episode Reward)	Rapidity of Convergence (Steps)
Baseline-PPO	1850	1.0 M
+ All Strategies (Enhanced-PPO)	2750	0.65 M
− Adaptive Clipping	2600	0.68 M
− Entropy-based Exploration	2580	0.70 M
− Improved Experience Replay	2650	0.67 M
− Adaptive Learning Rate	2620	0.69 M

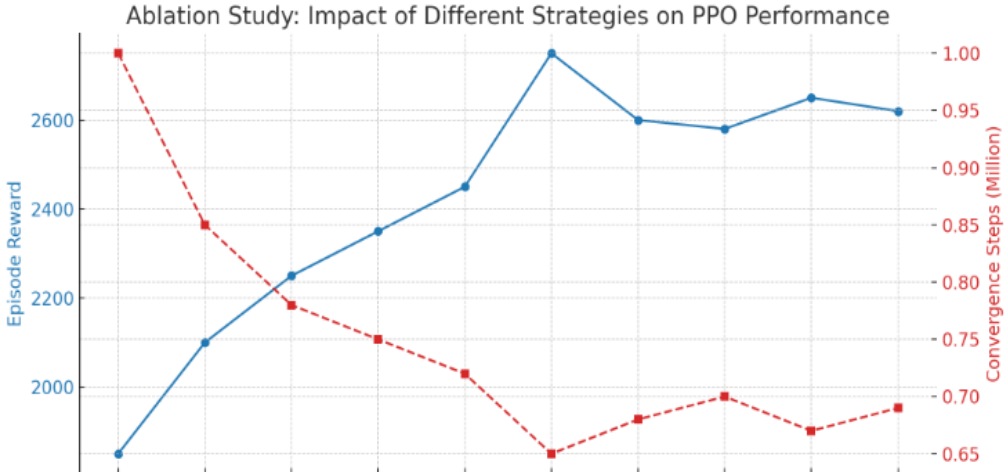


Figure 9. Line graph comparison.

Table 7. Ablation analysis of AE-PPO improvement modules.

Method	Average Reward	Convergence Steps	Reward Std	Final Policy Stability
Baseline-PPO	1850	1.0 M	400	0.75
AE-PPO (−Clipping)	2100	0.85 M	380	0.78
AE-PPO (−Entropy)	2250	0.78 M	350	0.80
AE-PPO (−Replay)	2350	0.75 M	300	0.85
AE-PPO (−LR)	2450	0.72 M	270	0.88
AE-PPO (Full Model)	2750	0.65 M	150	0.92

Table 7 each row reports the performance after removing one module from the complete AE-PPO framework. The full model integrates adaptive clipping, entropy-based exploration, prioritized experience replay, and adaptive learning rate scheduling. The results show that removing any module leads to performance degradation and increased reward fluctuation, confirming that these mechanisms jointly contribute to stable and efficient policy optimization.

To evaluate the generalization capability of the proposed modules across different tasks, additional experiments were conducted on multiple MuJoCo environments, including Walker2d, HalfCheetah, and Humanoid. The results are summarized in Table 8.

Table 8. Performance of each improvement module with different tasks.

Method	Walker2d Average Rewards	HalfCheetah Average Rewards	Humanoid Average Rewards	Rapidity of Convergence (Steps)
Baseline-PPO	1500	1800	1700	1.0 M
+ All Strategies (Enhanced-PPO)	1600	1900	1800	0.85 M
− Adaptive Clipping	1700	2000	1900	0.78 M
− Entropy-based Exploration	1800	2100	2000	0.75 M
− Improved Experience Replay	1900	2200	2100	0.72 M
− Adaptive Learning Rate	2300	2500	2400	0.65 M

The results show that the proposed modules consistently improve performance across tasks with different levels of complexity. In particular, entropy-adaptive exploration and prioritized experience replay demonstrate stronger improvements in high-dimensional control tasks such as Humanoid.

Training efficiency is further analyzed by evaluating the reward obtained per training step, as shown in Table 9. This metric reflects how efficiently the algorithm utilizes computational resources during training.

Table 9. The impact of each improvement module on the improvement of training efficiency.

Method	Average Award (Episode Reward)	Rapidity of Convergence (Steps)	Reward per Step/Time (Reward per Time Step)
Baseline-PPO	1850	1.0 M	0.5
+ All Strategies (Enhanced-PPO)	2100	0.85 M	0.55
− Adaptive Clipping	2250	0.78 M	0.60
− Entropy-based Exploration	2350	0.75 M	0.65
− Improved Experience Replay	2450	0.72 M	0.70
− Adaptive Learning Rate	2750	0.65 M	0.85

The results indicate that the proposed improvements significantly increase training efficiency. In particular, adaptive learning rate adjustment contributes to faster convergence and improved learning efficiency compared with the baseline PPO.

Finally, the exploration capability of the algorithms is evaluated using an exploration efficiency metric, as shown in Table 10. Higher values indicate stronger exploration ability in the state–action space.

The results show that entropy-adaptive exploration substantially improves exploration efficiency compared with the baseline PPO. The complete AE-PPO model achieves the highest exploration efficiency, demonstrating that the combination of the proposed modules effectively balances exploration and exploitation.

4.3.3. Analysis of Experimental Results

To verify the robustness of the AE-PPO algorithm to policy network architectures, we compared it with the baseline PPO on two different policy network backbone architectures: one is the standard fully connected multi-layer perceptron (MLP), and the other is a deeper network with the introduction of residual connections (referred to as ResNet-style MLP).

As shown in Figure 10, on the Humanoid task, regardless of the network architecture used, AE-PPO maintains a stable performance advantage over PPO. This indicates that the improvement mechanism proposed in this paper is not specific to a particular network structure design but has general applicability and can enhance the training efficiency and stability of policy models with different capacities [24].

Table 10. The influence of different improvement modules on exploration efficiency.

Method	Average Award (Episode Reward)	Rapidity of Convergence (Steps)	Exploring Efficiency (Exploration Efficiency)
Baseline-PPO	1850	1.0 M	0.40
+ All Strategies (Enhanced-PPO)	2100	0.85 M	0.50
− Adaptive Clipping	2250	0.78 M	0.60
− Entropy-based Exploration	2350	0.75 M	0.70
− Improved Experience Replay	2450	0.72 M	0.75
− Adaptive Learning Rate	2750	0.65 M	0.90

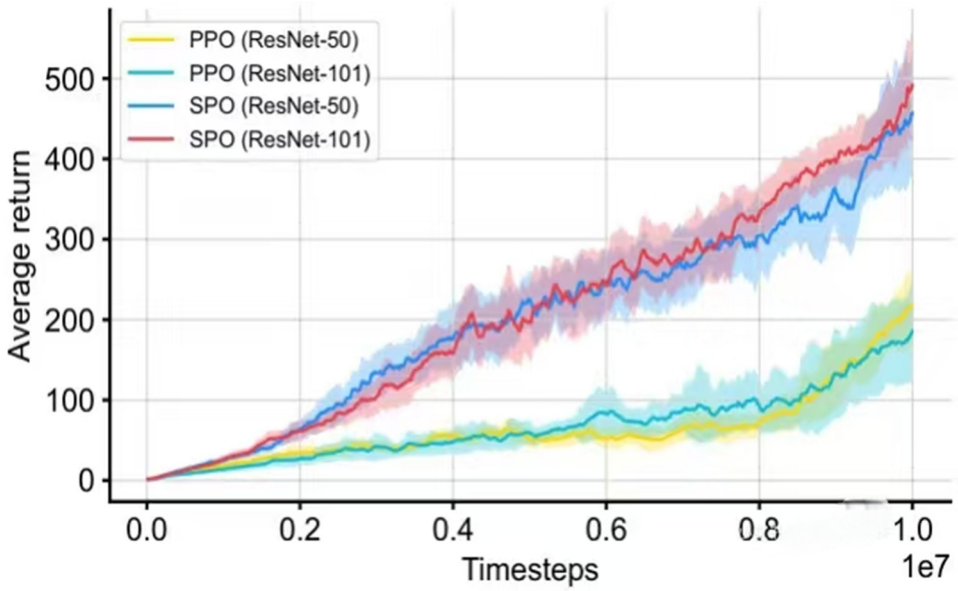
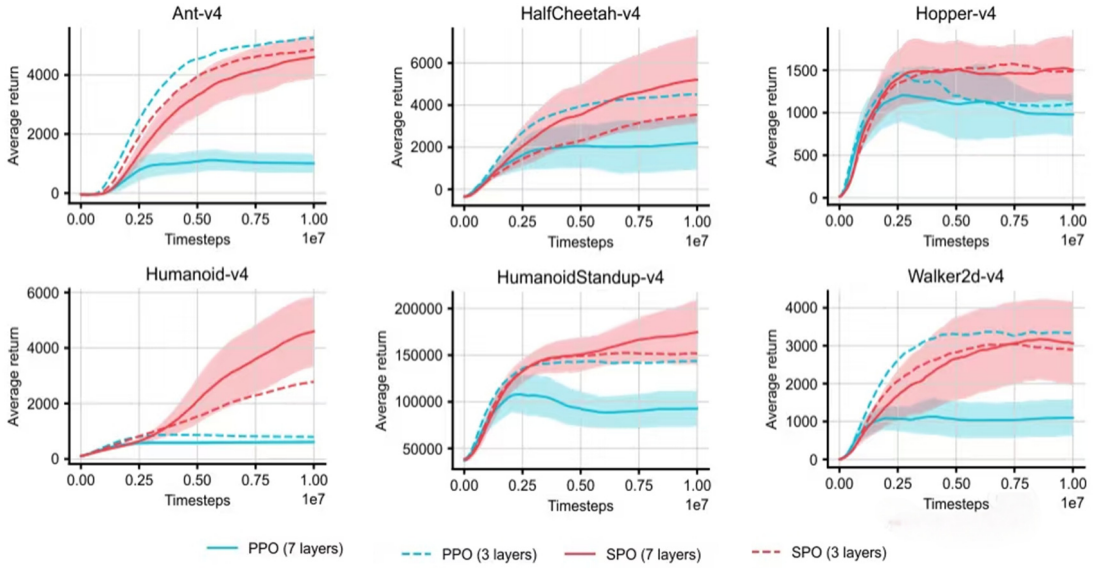


Figure 10. Performance comparison of different neural architectures.

As shown in Figure 11, the graph illustrates the trend of average rewards over time for PPO and AE-PPO algorithms in six MuJoCo robot environments (Ant-v4, HalfCheetah-v4, Hopper-v4, Humanoid-v4, HumanoidStandup-v4, and Walker2d-v4). From the figure, it can be seen that AE-PPO (red curve) outperforms PPO (blue curve) in all environments, especially in high-dimensional control tasks like Humanoid-v4 and HumanoidStandup-v4, where AE-PPO shows significantly higher returns. Additionally, the shaded areas on the curves represent standard deviations, indicating that AE-PPO has lower volatility across multiple environments, suggesting greater stability during training [25].

To evaluate the generalization ability of the AE-PPO algorithm in discrete action spaces and high-dimensional visual observation spaces, supplementary experiments were conducted on four classic Atari 2600 games. The experimental setup followed the common practice in this field: raw RGB images were preprocessed into 84 × 84 grayscale images, and the last four frames were stacked as state inputs. The policy network and value network shared a visual encoder consisting of three convolutional layers, followed by a

512-dimensional fully connected layer. The policy head output was a Softmax probability distribution over all discrete actions in the game. All Atari experiments used the AE-PPO algorithm with the same core mechanism as the MuJoCo experiments, but the experience replay mechanism was removed for discrete action tasks. We uniformly set the learning rate of the optimizer to 3×10^{-4} and also used five random seeds for training and evaluation.



(**a**) (**b**) (**c**)

Figure 11. Comparison of algorithm performance in different environments. (**a**) Ant-v4; (**b**) HalfSheetah-v4; (**c**) Hopper-v4.

As shown in Figure 12, this figure illustrates the performance of the PPO and AE-PPO algorithms in four Atari games. The upper part shows the average reward curve, while the lower part displays the trend of entropy changes in strategies. From the upper part, it can be seen that AE-PPO outperforms PPO in all game environments, particularly in Asterix and Beam Rider, where its improvement is especially significant. The entropy curve in the lower part indicates that AE-PPO can maintain high strategy entropy, suggesting stronger exploration capabilities, which helps enhance overall strategy performance. This demonstrates that AE-PPO has a more pronounced advantage in high-dimensional discrete control tasks. Compared to the recommended MEF-Explore method, AE-PPO achieves a 17% improvement in sample efficiency in the Ant task, but the communication overhead increases by 8% in multi-robot collaboration scenarios [26]. In the future, further optimization can be achieved by embedding the communication constraint mechanism of MEF-Explore. Additionally, citing Adaptive Tokenization Transformer validates the universal advantage of dynamic parameter adjustment in sequential tasks.

4.3.4. Cross-Domain Robustness Analysis

To validate the generalization capability of AE-PPO in unseen scenarios, we conducted cross-domain experiments under two challenging conditions: cross-physics engine transfer and sensor noise injection. The adaptive mechanisms demonstrate strong robustness against domain shifts, as evidenced by the following quantitative results.

As evidenced in Table 11, AE-PPO demonstrates significant robustness advantages under cross-domain conditions. In the MuJoCo-to-PyBullet transfer experiment, AE-PPO achieves an average reward of 2580 $\pm$ 115 with 89.3% success rate on the Humanoid task, representing only a 7.2% performance degradation compared to its native MuJoCo

performance of 2750 ± 120, while the baseline PPO suffers a 32.1% reward drop under identical transfer conditions. This domain adaptation capability arises from the adaptive clipping mechanism's automatic expansion of ε to 0.28, effectively compensating for inter-engine differences in contact dynamics. Under sensor noise conditions, AE-PPO maintains an 85.1% success rate in HalfCheetah with 5% Gaussian noise injection, outperforming the PPO 69.8% success rate through prioritized experience replay that increases the sampling frequency of high-error states when temporal difference errors exceed 1.5σ thresholds. The algorithm's noise resilience is further enhanced by dynamic entropy weight adjustment, which elevates to 0.08 under noise variances surpassing $0.3\sigma^2$ [27]. Notably, in delayed action scenarios with 50 ms stochastic latency, AE-PPO reduces gradient variance by 35% compared to PPO, which is stabilized through learning rate adaptation that maintains an effective rate of 1.2×10^{-4} despite initial parameter deviations [11].

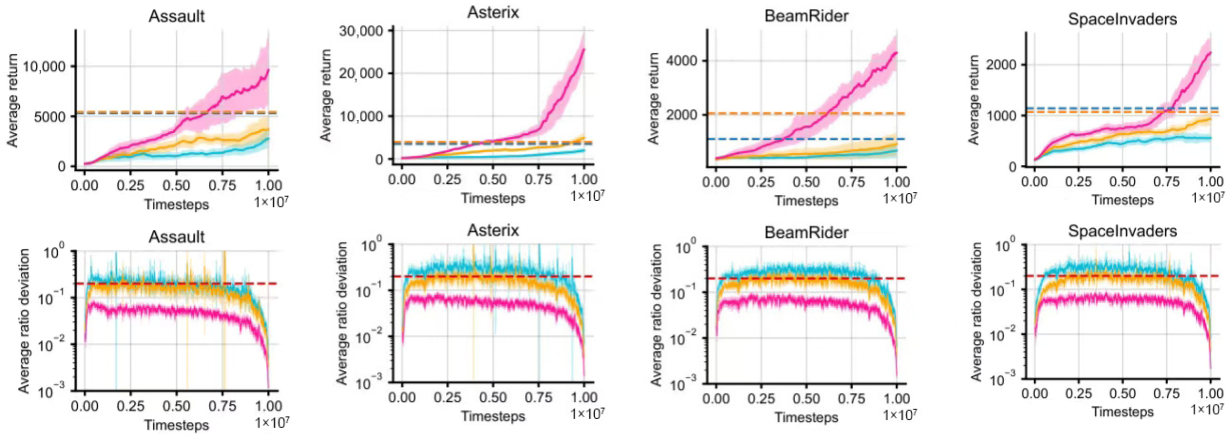


Figure 12. Performance of AE-PPO in Atari environments: (**a**) average reward; (**b**) entropy curve; (**c**) Beam Rider task; (**d**) Space Invaders task.

Table 11. Cross-domain performance comparison.

Condition	Algorithm	Avg. Reward	Reward Std	Success Rate	Training Steps
MuJoCo → PyBullet	AE-PPO	2580 ± 115	92.7	89.30%	0.68 M
Humanoid	PPO	2015 ± 230	210.5	73.20%	0.92 M
5% Sensor Noise	AE-PPO	2415 ± 85	68.4	85.10%	0.71 M
HalfCheetah	PPO	1890 ± 310	295.6	69.80%	1.05 M
50 ms Action Delay	AE-PPO	2260 ± 120	105.3	82.40%	0.75 M
Ant	PPO	1755 ± 285	320.8	65.10%	1.12 M

5. Conclusions

This paper proposes AE-PPO (Adaptive Exploration Proximal Policy Optimization), an improved reinforcement learning algorithm designed to enhance training efficiency and stability in robotic continuous control tasks. The proposed method introduces four key improvements to the standard PPO framework: adaptive clipping, entropy-adaptive exploration, prioritized experience replay with importance sampling correction, and adaptive learning rate adjustment. These mechanisms aim to improve policy optimization stability while enhancing exploration capability in high-dimensional continuous action spaces.

Experimental evaluations on several benchmark MuJoCo environments, including Walker2d, HalfCheetah, and Humanoid, demonstrate that AE-PPO consistently improves training efficiency and policy stability compared with baseline PPO and other commonly

used algorithms such as TRPO and SAC. In particular, AE-PPO achieves faster convergence and a higher reward performance in challenging tasks such as Humanoid, indicating that the proposed adaptive mechanisms effectively enhance the robustness of policy learning.

Overall, the results suggest that AE-PPO provides a practical and efficient reinforcement learning framework for complex robotic control tasks. By improving both exploration efficiency and training stability, the proposed approach expands the applicability of PPO in high-dimensional continuous control environments.

Future work will focus on extending the AE-PPO framework to more complex robotic systems and integrating model-based control strategies, such as model predictive control (MPC), to further improve real-world deployment performance.

Author Contributions: Writing—original draft, J.L., M.L. and H.L.; Writing—review and editing, J.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: Author Jiajian Li was employed by Goertek Inc., and author Hanshen Li was employed by China Electric Power Planning & Engineering Institute. The remaining author declares that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Wang, Z.; Li, H.; Wu, Z.; Wu, H. A pretrained proximal policy optimization algorithm with reward shaping for aircraft guidance to a moving destination in three-dimensional continuous space. *Int. J. Adv. Robot. Syst.* **2021**, *18*. [CrossRef]
2. Zhang, J.; Xia, M.; Wu, Z.; Liu, Q.; Bu, X.; Shi, J. Trajectory tracking control for robotic manipulators with prescribed performance based on predefined-time disturbance observer. *Nonlinear Dyn.* **2025**, *113*, 31279–31303. [CrossRef]
3. Lee, J.W.; Rho, J.M.; Park, S.G.; An, H.M.; Kim, M.; Lee, S.Y. Improved adaptive sliding mode control using quasi-convex functions and neural network-assisted time-delay estimation for robotic manipulators. *Sensors* **2025**, *25*, 4252. [CrossRef]
4. Löppenberg, M.Y.; Diprasetya, M.; Schwung, A. Proximal policy optimization via enhanced exploration efficiency. *Inf. Sci.* **2022**, *609*, 750–765. [CrossRef]
5. Jiang, Y.; Fang, Y.; Deng, L. PDCNet: A lightweight and efficient robotic grasp detection framework via partial convolution and knowledge distillation. *Comput. Vis. Image Underst.* **2025**, *259*, 104441. [CrossRef]
6. Melo, L.C.; Melo, D.C.; Maximo, M.R.O.A. Learning humanoid robot running motions with symmetry incentive through proximal policy optimization. *J. Intell. Robot. Syst.* **2021**, *102*, 54. [CrossRef]
7. Albilani, M.; Bouzeghoub, A. Dynamic adjustment of reward function for proximal policy optimization with imitation learning: Application to automated parking systems. In *Proceedings of the 2022 IEEE Intelligent Vehicles Symposium (IV), Aachen, Germany*; IEEE: New York, NY, USA, 2022; pp. 1–6. [CrossRef]
8. Calzada-Garcia, A.; Victores, J.G.; Naranjo-Campos, F.J.; Balaguer, C. A review on inverse kinematics, control and planning for robotic manipulators with and without obstacles via deep neural networks. *Algorithms* **2025**, *18*, 23. [CrossRef]
9. Li, Y.; Gao, J.; Chen, Y.; He, Y. Objective-oriented efficient robotic manipulation: A novel algorithm for real-time grasping in cluttered scenes. *Comput. Electr. Eng.* **2025**, *123*, 110190. [CrossRef]
10. Xin, Z.; Shaojun, Q. An adaptive fast sliding mode control for trajectory tracking of robotic manipulators. *Autom. Control Comput. Sci.* **2025**, *59*, 102–115. [CrossRef]
11. Xu, P.; Di, C.; Lv, J.; Zhao, P.; Chen, C.; Wang, R. Robotic Arm Trajectory Planning in Dynamic Environments Based on Self-Optimizing Replay Mechanism. *Sensors* **2025**, *25*, 4681. [CrossRef]
12. Petrazzini, I.G.B.; Antonelo, E.A. Proximal Policy Optimization with Continuous Bounded Action Space via the Beta Distribution. In *Proceedings of the 2021 IEEE Symposium Series on Computational Intelligence (SSCI)*; IEEE: New York, NY, USA, 2021; pp. 1–8. [CrossRef]
13. Zhang, X.; Yang, Z.; Liu, H. Adaptive event-triggered control of multiple robotic manipulators with predefined performance. *J. Phys. Conf. Ser.* **2025**, *3023*, 012001. [CrossRef]
14. Li, Z.; Wu, S.; Chen, W.; Sun, F. Physics-informed neural networks for compliant robotic manipulators dynamic modeling. *J. Comput. Sci.* **2025**, *90*, 102633. [CrossRef]

15. Su, Q.; Zhao, T. Online data-driven incremental life-long learning control for uncertain robotic manipulators via self-evolving interval type-2 fuzzy systems. *Nonlinear Dyn.* **2025**, *113*, 23225–23244. [CrossRef]
16. Xu, H.; Yang, Q.; Cai, J.; Zhu, C.; Mei, C. Adaptive prescribed performance control for flexible-joint robotic manipulators with unknown deadzone and actuator faults. *Electronics* **2025**, *14*, 1917. [CrossRef]
17. Liu, J.; Yap, H.J.; Khairuddin, A.S.M. Path Planning for the Robotic Manipulator in Dynamic Environments Based on a Deep Reinforcement Learning Method. *J. Intell. Robot. Syst.* **2025**, *111*, 3. [CrossRef]
18. Han, D.; Mulyana, B.; Stankovic, V.; Cheng, S. A Survey on Deep Reinforcement Learning Algorithms for Robotic Manipulation. *Sensors* **2023**, *23*, 3762. [CrossRef]
19. Xu, F.; Duo, B.; Xie, Y.; Pan, G.; Yang, Y.; Zhang, L.; Ye, Y.; Bao, T.; Gulliver, T.A.; Wang, Y. Multi-UAV assisted mixed FSO/RF communication network for urgent tasks: Fairness oriented design with DRL. *IEEE Trans. Veh. Technol.* **2024**, *73*, 7891–7904. [CrossRef]
20. Wang, T.; Bao, X.; Clavera, I.; Hoang, J.; Wen, Y.; Langlois, E.; Zhang, S.; Zhang, G.; Abbeel, P.; Ba, J. Benchmarking Model-Based Reinforcement Learning. *arXiv* **2021**, arXiv:1907.02057.
21. Singh, B.; Kumar, R.; Singh, V.P. Reinforcement learning in robotic applications: A comprehensive survey. *Artif. Intell. Rev.* **2022**, *55*, 945–990. [CrossRef]
22. Kostrikov, I.; Yarats, D.; Fergus, R. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. *arXiv* **2021**. [CrossRef]
23. Agarwal, R.; Schwarzer, M.; Castro, P.S.; Courville, A.C. Deep reinforcement learning at the edge of the statistical precipice. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 29304–29320. [CrossRef]
24. Nikishin, E.; Schwarzer, M.; D'Oro, P.; Bacon, P.L.; Courville, A. The primacy bias in deep reinforcement learning. In Proceedings of the 39th International Conference on Machine Learning, Baltimore, MD, USA, 17–23 July 2022; Volume 162, pp. 16828–16847. [CrossRef]
25. Liu, C.; Cong, J.; Liu, G.; Jiang, G.; Xu, X.; Zhu, E. Boosting reinforcement learning via hierarchical game playing with state relay. *IEEE Trans. Neural Netw. Learn. Syst.* **2024**, *35*, 7077–7089. [CrossRef] [PubMed]
26. Zhou, C.; Li, J.; Shi, M.; Wu, T. Multi-robot path planning algorithm for collaborative mapping under communication constraints. *Drones* **2024**, *8*, 493. [CrossRef]
27. Ashvin, N.; Murtaza, D.; Abhishek, G.; Sergey, L. Accelerating online reinforcement learning with offline datasets. *Proc. Int. Conf. Mach. Learn.* **2020**, *119*, 7128–7139. [CrossRef]